



UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales

Electrónica Digital II

Informe de Laboratorio Nº2

Alumnos:
Gimenez, Galilea Candelaria
Ramos, Antonella Carolina
Salomon Oliden, Naiara Luz
Valencia Pezo, Luz Estrella

Córdoba,

1. Introducción

El objetivo del trabajo práctico consiste en diseñar e implementar un programa en lenguaje ensamblador para el microcontrolador PIC16F887, orientado a controlar un banco de ocho LEDs mediante dos modos de funcionamiento seleccionables: un contador binario incremental de 8 bits y una secuencia luminosa. El sistema permite alternar entre ambos comportamientos mediante un pulsador externo, que incorpora un algoritmo de antirrebote implementado por software para garantizar la estabilidad en la lectura de los estados lógicos.

Para la realización del proyecto se integran conceptos de hardware y software. A nivel físico, se configuran los pines del integrado como entradas o salidas digitales mediante la manipulación de los registros de control correspondientes y la desactivación de sus funciones analógicas. Asimismo, se aplican leyes eléctricas fundamentales, como la Ley de Ohm, para calcular las resistencias limitadoras de los LEDs y estabilizar los niveles de tensión de las entradas mediante la resistencia interna de pull-up para el pulsador de control y una resistencia externa de pull-up en la línea de reset (MCLR), evitando estados flotantes e indeterminados. El desarrollo del programa se lleva a cabo en lenguaje ensamblador utilizando el entorno MPLAB, profundizando en el uso de instrucciones, el registro de trabajo W y el manejo directo de puertos de entrada y salida.

2. Hardware

2.1. Asignación y justificación de pines

Módulo	Señal	Terminal del PIC
Comunicación serie	TX	RC6
Comunicación serie	RX	RC7
Pulsador	Entrada	RB0
Banco de LED	LED 0 a LED 7	RD0 a RD7
Displays	Segmentos a a g y dp	RC0 a RC5, RE0, RE1
Displays	Selección de dígitos	RA0, RA1, RA2, RA3
Teclado	Filas	RB4, RB5, RB6, RB7
Teclado	Columnas	RB1, RB2, RB3, RE2
Conversor ADC	Entrada analógica	RA5 (AN4)

Tabla 1: *Asignación de terminales.*

Función	Pin	Configuración
Pulsador	RB0	Entrada digital
LED0	RD0	Salida digital
LED1	RD1	Salida digital
LED2	RD2	Salida digital
LED3	RD3	Salida digital
LED4	RD4	Salida digital
LED5	RD5	Salida digital
LED6	RD6	Salida digital
LED7	RD7	Salida digital

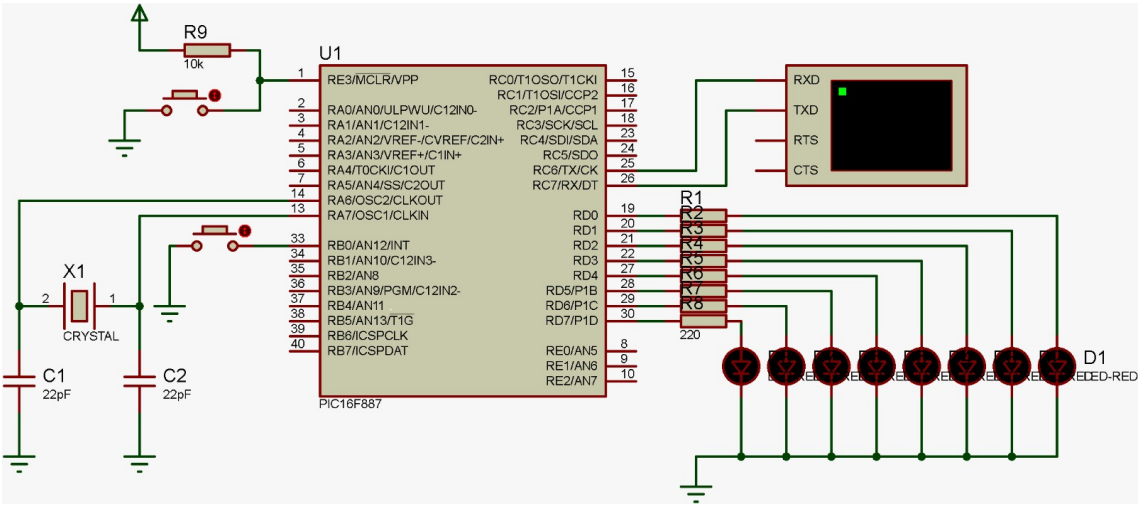
Tabla 2: *Asignación de pines utilizada.*

Justificación de la elección de pines:

Puerto D (RD0:RD7): Se eligió como puerto de salida digital completo para controlar el banco de 8 LEDs. Al ser un puerto de 8 bits sin funciones analógicas compartidas conflictivas.

Pin RB0: Se eligió como entrada digital para el pulsador por pertenecer al Puerto B, lo que permite habilitar su resistencia de Pull-Up interna configurable por software (WPUB0), sin necesidad de agregar más componentes externos en la placa física.

2.2. Diagrama de conexión



2.3. Cálculos eléctricos

Para limitar la corriente que circula a través de los LEDs, se calcula la resistencia necesaria mediante la siguiente expresión:

$$R = \frac{V_{CC} - V_F}{I_F}$$

Se consideran los siguientes parámetros:

- Tensión de alimentación: $V_{CC} = 5\,\text{V}$
- Caída de tensión del LED rojo: $V_F = 2\,\text{V}$
- Corriente deseada: $I_F = 0{,}014\,\text{A}$ $(14\,\text{mA})$, seleccionada para asegurar un encendido visible sin sobrecargar las salidas del microcontrolador.

Sustituyendo:

$$R = \frac{5\,\text{V} - 2\,\text{V}}{0{,}014\,\text{A}} \approx 214{,}29\,\Omega \implies R \approx 220\,\Omega$$

Por lo tanto, se adopta el valor comercial estándar más próximo de $220\,\Omega$.

Con dicha resistencia, la corriente teórica efectiva es:

$$I = \frac{5\,\mathrm{V} - 2\,\mathrm{V}}{220\,\Omega} \approx 13{,}64\,\mathrm{mA}$$

Este valor cumple con las especificaciones del PIC16F887, cuyo límite máximo absoluto por pin de E/S es de 25 mA y de corriente combinada para los puertos C y D (máximo de 200 mA).

3. Software

3.1. Descripción del algoritmo

El programa de control del microcontrolador se organiza mediante una fase de inicialización de registros y un lazo de ejecución infinito estructurado como una máquina de estados:

1. Fase de Inicialización: Mediante la limpieza de los registros ANSEL y ANSELH, se deshabilitan las funciones analógicas de los pines para asegurar un funcionamiento digital. Posteriormente, se configura el puerto D como salida para controlar los LEDs, y el pin RB0 como entrada para registrar el estado del pulsador.

2. Sondeo (Polling) y Antirrebote: El sistema realiza un sondeo continuo del pin RB0. Ante una transición a nivel bajo, se invoca una subrutina de retardo por software para filtrar rebotes mecánicos y asegurar una lectura estable. Una vez confirmado el estado, el programa mide el tiempo de presión física del pulsador y bloquea la ejecución en un lazo de espera de liberación para evitar falsas activaciones consecutivas.

3. Lógica del Algoritmo : se basa en una máquina de dos estados seleccionables mediante la duración temporal del accionamiento del pulsador único:
   - Gestión del Contador (Estado 0): Es el modo activo por defecto en el arranque. El microcontrolador registra las pulsaciones rápidas (menores a 3 segundos) para incrementar secuencialmente un registro contador , reflejando su valor binario en el puerto PORTD. Si el usuario mantiene presionado el botón de forma sostenida (3 segundos o más), el programa interpreta este evento como una orden de cambio de modo y conmuta la máquina de estados hacia el modo dinamico de LEDs.
   - Gestión de la Secuencia Luminosa (Estado 1): En este modo, el sistema ejecuta un juego de luces dinámico basado en desplazamientos de bits. El pulsador altera el flujo luminoso según tres rangos temporales bien definidos: una pulsación breve de hasta 1.5 segundos se utiliza como conmutador de pausa/reproducción; un accionamiento intermedio de entre 1.5 y 3 segundos cambia el movimiento en los LEDs; y una pulsación prolongada de más de 3 segundos actúa como un reinicio general que devuelve al sistema al modo de contador binario (Estado 0).

3.2. Diagrama de flujo

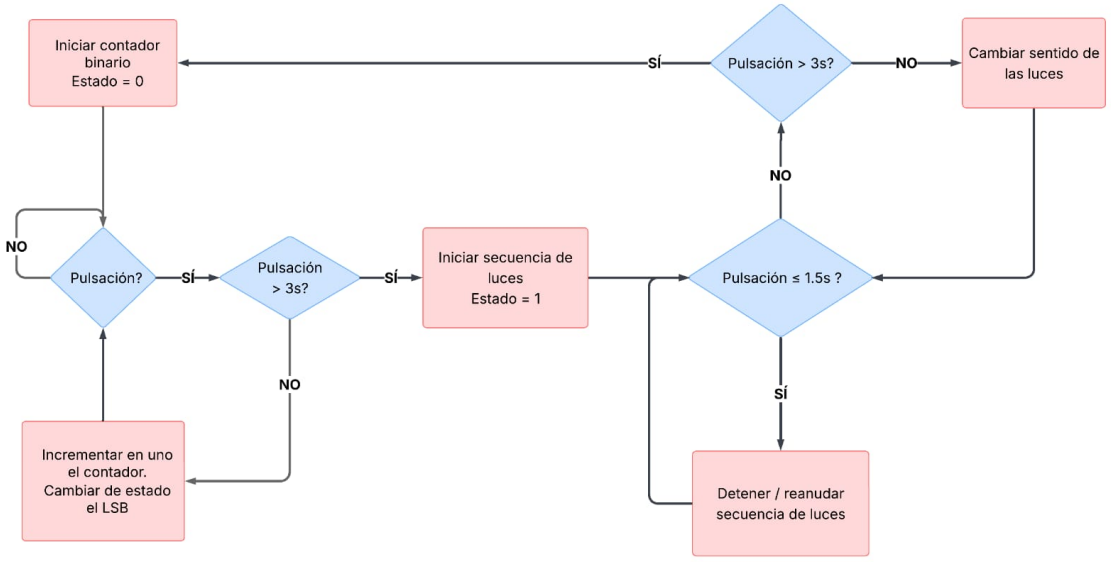


Figura 1: *Diagrama de flujo*

3.3. Análisis temporal de rutinas

Tomando como base la subrutina `RETARDO_5MS` junto con la llamada `CALL`, de la cual derivan los demas tiempos deterministas del programa.

Para el PIC16F887, considerando el cristal de 4 MHz utilizado en el trabajo, determinar primero el período del ciclo de instrucción mediante

$$T_{CY} = \frac{4}{F_{OSC}}.$$

$$T_{CY} = \frac{4}{4 \times 10^6 \text{ Hz}} = 1 \ \mu\text{s}$$

Cada ciclo de instruction toma exactamente

$$1 \ \mu\text{s}$$

Las instrucciones comunes tardan 1 ciclo (1 μs), mientras que las instrucciones que producen un salto de flujo tardan dos ciclos (2 μs).

Segmento `RETARDO_5MS`:

Se analiza la ejecución completa desde la llamada `CALL` hasta el retorno `RETURN`. Esta rutina no depende de que se presione o suelte un botón, simplemente quema tiempo mediante bucles decrecientes.

```
            CALL    RETARDO_5MS    ; [2 ciclos]
; ------------------------------------------
RETARDO_5MS:
            MOVLW   d'65'          ; [1 ciclo]
            MOVWF   CONT1          ; [1 ciclo]
R_5MS_LOOP:
            MOVLW   d'25'          ; [1 ciclo]
            MOVWF   CONT2          ; [1 ciclo]
            DECFSZ  CONT2, F       ; [1 ciclo si no salta, 2 ciclos si salta]
            GOTO    $-1            ; [2 ciclos]
            DECFSZ  CONT1, F       ; [1 ciclo si no salta, 2 ciclos si salta]
            GOTO    R_5MS_LOOP     ; [2 ciclos]
            RETURN                 ; [2 ciclos]
```

Cálculo detallado de ciclos

1. **Llamada e inicialización:**
   - `CALL RETARDO_5MS`: 2 ciclos.
   - `MOVLW d'65'`: 1 ciclo.
   - `MOVWF CONT1`: 1 ciclo.

   Subtotal inicial: $2 + 1 + 1 = 4$ ciclos.

2. **Bucle interno (`CONT2` cargado con 25):**
   - Carga: `MOVLW d'25'` (1 ciclo) + `MOVWF CONT2` (1 ciclo) = 2 ciclos.
   - Primeras 24 iteraciones: `DECFSZ` no salta (1 ciclo) + `GOTO $-1` (2 ciclos) = 3 ciclos $\times$ 24 = 72 ciclos.
   - Última iteración ($CONT2 = 1 \rightarrow 0$): `DECFSZ` salta al completarse el decremento, insumiendo 2 ciclos.

   Total del bucle interno por iteración de `CONT1`:

$$N_{\text{interno}} = 2 + 72 + 2 = 76 \text{ ciclos}$$

3. **Bucle externo (`CONT1` cargado con 65):**
   - Primeras 64 iteraciones: Ejecutan el bucle interno completo más la evaluación sin salto del bucle externo:

$$76 + 1 \text{ (DECFSZ)} + 2 \text{ (GOTO)} = 79 \text{ ciclos} \implies 64 \times 79 = 5056 \text{ ciclos}$$

- Última iteración (65ª): Ejecuta el bucle interno y luego `DECFSZ` salta al llegar a cero insumiendo 2 ciclos:

$$76 + 2 \text{ (DECFSZ)} = 78 \text{ ciclos}$$

Total del bucle externo:

$$N_{\text{externo}} = 5056 + 78 = 5134 \text{ ciclos}$$

4. **Retorno de subrutina:**
   - `RETURN`: 2 ciclos.

Tiempo total obtenido

Sumando todas las fases de ejecución:

$$N_{\text{total}} = 4 \text{ (inicio)} + 5134 \text{ (bucles)} + 2 \text{ (retorno)} = 5140 \text{ ciclos}$$

Multiplicando por el período del ciclo de instrucción ($T_{CY} = 1\ \mu$s):

$$T_{\text{total}} = 5140 \times 1\ \mu\text{s} = 5140\ \mu\text{s} = 5{,}14 \text{ ms}$$

Extrapolación a rutinas derivadas

Considerando que las demás rutinas de temporización determinista invocan de manera consecutiva a la rutina base, sus tiempos de ejecución resultan:

- **Antirrebote (`RETARDO_20MS`):** Ejecuta 4 llamadas a `RETARDO_5MS` con overhead de llamada y retorno:

$$N = 2 \text{ (CALL)} + (4 \times 5140) + 2 \text{ (RETURN)} = 20564 \text{ ciclos} \implies T = 20{,}564 \text{ ms}$$

- **Medición de pulsación (`RETARDO_100MS`):** Ejecuta 5 llamadas a `RETARDO_20MS`:

$$N = 2 \text{ (CALL)} + (5 \times 20564) + 2 \text{ (RETURN)} = 102824 \text{ ciclos} \implies T = 102{,}824 \text{ ms}$$

3.4. Código fuente

```
list p=16f887
include "P16f887.INC"

    __CONFIG _CONFIG1, _FOSC_XT & _WDTE_OFF & _PWRTE_ON & _MCLRE_ON
    __CONFIG _CONFIG2, _LVP_OFF & _BOR_OFF

; vector de reset
ORG     0x00
GOTO    main

;  mapa de memoria
SALIDA_D    EQU 0x20    ; contador
ESTADO      EQU 0x21    ; 0 = contador, 1 = secuencia de luces
TIEMPO      EQU 0x22    ; mide duración de pulsación (en bloques de 100ms)
SEC_LUCES   EQU 0x23    ; secuencia de los LEDs
DIR_LUCES   EQU 0x24    ; 0 = izquierda, 1 = derecha
PAUSA       EQU 0x25    ; 0 = activo, 1 = detenido
CONT1       EQU 0x26    ; contador para el bucle de retardo
CONT2       EQU 0x27    ; c0ntador secundario de retardo
CONT_SEC    EQU 0x28    ; contador para velocidad de luces

ORG 0x20
main:
    ; deshabilitar entradas analogicas (todas a digital)
    BANKSEL ANSEL
    CLRF    ANSEL
    CLRF    ANSELH

    BANKSEL PORTD
    CLRF    PORTD

    BANKSEL TRISD
    CLRF    TRISD       ; PORTD como salida
    BSF     TRISB, 0    ; RB0 como entrada

    ; habilitar pullUp interna en RB0
    BANKSEL OPTION_REG
    BCF     OPTION_REG, 7
    BANKSEL WPUB
    BSF     WPUB, 0
```

```
    ;inicializar variables (Estado 0)
    BANKSEL SALIDA_D
    CLRF    SALIDA_D
    CLRF    ESTADO
    MOVLW   d'100'
    MOVWF   CONT_SEC    ; velocidad inicial de la secuencia

; linea principal del flujo
LINEA_PRINCIPAL:
    ; revisar si hay presión en el botón
    BANKSEL PORTB
    BTFSS   PORTB, 0    ; si es 0 (presionado), salta a leer tiempo
    GOTO    PROCESAR_PULSACION

    ; si no hay presión, procesar secuencia (solo si estado = 1)
    BANKSEL ESTADO
    BTFSS   ESTADO, 0
    GOTO    LINEA_PRINCIPAL ; si es estado 0, solo esperar botón

    BTFSC   PAUSA, 0
    GOTO    LINEA_PRINCIPAL ; si está pausado, no mueve los LEDs

    ; contador de la secuencia
    DECFSZ  CONT_SEC, F
    GOTO    ESPERA_BOTON ; si no llegó a 0, sigue leyendo botón
    MOVLW   d'50'
    MOVWF   CONT_SEC       ; recarga la velocidad de los LEDs

    ; ejecutar desplazamiento
    BTFSC   DIR_LUCES, 0
    GOTO    MOVER_DERECHA

MOVER_IZQUIERDA:
    BCF     STATUS, C
    RLF     SEC_LUCES, F
    BTFSS   STATUS, C
    GOTO    ACTUALIZAR_PUERTO
    BSF     SEC_LUCES, 0   ; reinicia el bit si desbordó
    GOTO    ACTUALIZAR_PUERTO

MOVER_DERECHA:
```

```
    BCF     STATUS, C
    RRF     SEC_LUCES, F
    BTFSS   STATUS, C
    GOTO    ACTUALIZAR_PUERTO
    BSF     SEC_LUCES, 7    ; reinicia el bit si desbordó

ACTUALIZAR_PUERTO:
    MOVF    SEC_LUCES, W
    BANKSEL PORTD
    MOVWF   PORTD

ESPERA_BOTON:
    CALL    RETARDO_5MS     ; espera para que el programa no corra tan rápido
    GOTO    LINEA_PRINCIPAL

; lógica de pulsación (filtro antirrebote y mediciónes)
PROCESAR_PULSACION:
    CALL    RETARDO_20MS    ; antirrebote
    BANKSEL PORTB
    BTFSC   PORTB, 0
    GOTO    LINEA_PRINCIPAL ; retorna si fue ruido mecánico

    BANKSEL TIEMPO
    CLRF    TIEMPO

MANTENIENDO_PULSACION:
    BANKSEL PORTB
    BTFSC   PORTB, 0        ; verifica si se soltó el botón
    GOTO    EVALUAR_FLUJO   ; si se soltó, evalúa las decisiones

    CALL    RETARDO_100MS
    BANKSEL TIEMPO
    INCF    TIEMPO, F
    MOVF    TIEMPO, W
    SUBLW   d'40'           ; límite de 4s para evitar desbordamiento
    BTFSC   STATUS, Z
    DECF    TIEMPO, F       ; mantiene el tiempo estancado en 40
    GOTO    MANTENIENDO_PULSACION

EVALUAR_FLUJO:
    BANKSEL ESTADO
    BTFSC   ESTADO, 0
```

```
    GOTO    EVALUAR_ESTADO_1

EVALUAR_ESTADO_0:
    ; ¿pulsación >= 3s (30 décimas)?
    MOVF    TIEMPO, W
    SUBLW   d'29'
    BTFSS   STATUS, C       ; C=0 si TIEMPO >= 30
    GOTO    IR_A_ESTADO_1

    ; pulsación (< 3s): incrementa contador
    INCF    SALIDA_D, F
    MOVF    SALIDA_D, W
    BANKSEL PORTD
    MOVWF   PORTD
    GOTO    LINEA_PRINCIPAL

EVALUAR_ESTADO_1:
    ; ¿pulsación >= 3s?
    MOVF    TIEMPO, W
    SUBLW   d'29'
    BTFSS   STATUS, C
    GOTO    IR_A_ESTADO_0

    ; ¿pulsación <= 1.5s (15 décimas)?
    MOVF    TIEMPO, W
    SUBLW   d'15'
    BTFSC   STATUS, C       ; C=1 si TIEMPO <= 15
    GOTO    CAMBIAR_PAUSA

    ; pulsación entre 1.5s y 3s: cambiar dirección
    MOVLW   0x01
    XORWF   DIR_LUCES, F    ; invierte el estado de la dirección
    GOTO    LINEA_PRINCIPAL

CAMBIAR_PAUSA:
    MOVLW   0x01
    XORWF   PAUSA, F        ; cambia de 0 a 1 o de 1 a 0 para pausar
    GOTO    LINEA_PRINCIPAL

;transciciones de estados
IR_A_ESTADO_1:
    BSF     ESTADO, 0       ; pasa a Estado = 1
```

```
    MOVLW   0x01
    MOVWF   SEC_LUCES       ; inicia LED en el primer bit
    CLRF    DIR_LUCES       ; dirección izquierda
    CLRF    PAUSA           ; arranca sin pausa
    MOVF    SEC_LUCES, W
    BANKSEL PORTD
    MOVWF   PORTD
    GOTO    LINEA_PRINCIPAL

IR_A_ESTADO_0:
    CLRF    ESTADO          ; vuelve al Estado = 0
    CLRF    SALIDA_D        ; reinicia contador binario
    MOVF    SALIDA_D, W
    BANKSEL PORTD
    MOVWF   PORTD
    GOTO    LINEA_PRINCIPAL

; subrutinas de retardo (4MHz)
RETARDO_5MS:
    MOVLW   d'65'
    MOVWF   CONT1
R_5MS_LOOP:
    MOVLW   d'25'
    MOVWF   CONT2
    DECFSZ  CONT2, F
    GOTO    $-1
    DECFSZ  CONT1, F
    GOTO    R_5MS_LOOP
    RETURN

RETARDO_20MS:
    CALL    RETARDO_5MS
    CALL    RETARDO_5MS
    CALL    RETARDO_5MS
    CALL    RETARDO_5MS
    RETURN

RETARDO_100MS:
    CALL    RETARDO_20MS
    CALL    RETARDO_20MS
    CALL    RETARDO_20MS
    CALL    RETARDO_20MS
```

```
CALL    RETARDO_20MS
RETURN

END
```

4. Pruebas y resultados

Con el fin de verificar el funcionamiento del circuito y del código, se probó el sistema de manera ordenada sobre la placa. El objetivo principal fue asegurar que respondiera correctamente según el tiempo que se mantuviera presionado el botón.

Los resultados de estas pruebas se resumen en la siguiente tabla:

Prueba	**Resultado esperado**	**Resultado observado**
Accionamiento del pulsador	Al presionar el pulsador en RB0 (activo en bajo), el pin debe pasar a 0 V. Al soltarlo, la resistencia de pull-up debe restablecer instantáneamente los 5 V .	Las mediciones con multímetro confirmaron transiciones limpias entre 0 V y 5 V .
Filtro Antirrebote	El retardo por software debe absorber los ruidos mecánicos del contacto. Al presionar el botón una vez, el programa debe registrar un único evento sin saltos aleatorios.	La transición de estados y los incrementos del contador se realizan de a una unidad por pulsación, evidenciando un filtrado eficaz.
Visualización del conteo binario	En el Estado 0, cada pulsación corta (< 3 s) debe incrementar en uno un registro RAM y reflejar su valor en binario en los LEDs del puerto D.	Se verificó la progresión binaria completa desde `00000000` hasta `11111111`. El uso de resistencias de 220 Ω garantizó un brillo de alta intensidad y un funcionamiento dinámico correcto.
Secuencia de LEDs y control temporal	Al presionar por más de 3 s, debe conmutar al Estado 1 (luces). Una vez allí: pulsaciones $\leq 1{,}5$ s deben pausar la secuencia; entre 1,5 s y 3 s deben invertir el giro; y ≥ 3 s deben resetear al Estado 0.	El algoritmo midió con precisión los tiempos de pulsación. Todas las funciones dinámicas de la secuencia y el retorno al contador respondieron según lo diseñado.

Tabla 3: *Resumen de las pruebas realizadas.*

Dificultades: El programa no iniciaba al encender el microcontrolador: Al cargar el código en el chip, el circuito no realizaba ninguna acción y los LEDs quedaban apagados. Revisando el código fuente, nos dimos cuenta de que faltaba la directiva ORG, por lo que el microcontrolador no encontraba el punto de arranque de las

instrucciones. Al agregar esa línea al inicio del programa y volver a grabarlo, el sistema empezó a funcionar correctamente.

5. Conclusiones

A través del desarrollo de este trabajo práctico se logró diseñar, implementar y verificar un sistema basado en el microcontrolador PIC16F887, capaz de operar como un contador binario de 8 bits y un secuenciador de luces dinámico. El proyecto permitió consolidar de manera práctica la integración de periféricos de entrada/salida digitales en lenguaje ensamblador, comprendiendo que el software y el hardware deben actuar de forma coordinada para obtener un funcionamiento estable.

Durante la puesta en marcha, la resolución de fallas de software resultó una instancia fundamental de aprendizaje técnico y de la importancia de las directivas para instruir al microcontrolador.Se comprendió que el tratamiento del pulsador no se limita a un retardo antirrebote, sino que incluye la medición del tiempo que permanece presionado. Gracias a esto, fue posible realizar múltiples acciones con un único botón: incrementar la cuenta, pausar o reanudar la secuencia, cambiar el sentido de giro y alternar entre los modos de funcionamiento, demostrando la versatilidad que se puede lograr mediante la programación.

Propuestas de Mejora y Optimización

Para futuras revisiones del diseño, se propone la siguiente mejora:

- **Optimización del Prototipo Físico (Hardware):** Dado que el circuito actual se montó de forma temporal sobre una protoboard, la principal mejora de hardware consiste en soldarlo en una placa multipropósito. Esto garantizaría la eliminación de fallos por falsos contactos mecánicos y reduciría las capacitancias parásitas.
- Intercambiar la asignación de los pines `RB0` y `RE2`: destinar `RB0` como cuarta columna del teclado matricial y reubicar el pulsador en `RE2`. Esto permite concentrar todas las columnas en el puerto B, simplificando la rutina de lectura y la organización del código.